



ble-sensors-mqtt

Manuale di installazione e utilizzo

Version 1.0.0 - Edizione italiana

Alessandro De Salvo - EUPL-1.2

> Host supportati: Linux x86_64/arm64 su Debian/Ubuntu/Raspberry Pi OS e RHEL/Rocky/AlmaLinux/CentOS/Fedora, oltre a Windows 11+ e macOS Tahoe 26+. Bluetooth interno e USB sono supportati tramite lo stack del sistema operativo. Vedere `HOSTS.it.md`.

1. Scopo

`ble-sensors-mqtt 1.0.0` è un gateway multi-sensore per Raspberry Pi. I plugin acquisiscono dati BLE o Tuya Cloud e producono un formato comune con identificatore, nome, marca, modello, protocollo, segnale, timestamp e tutti i valori disponibili. Lo stesso snapshot alimenta MQTT, Prometheus e SNMP.

2. Installazione

```
sudo apt update
sudo apt install -y bluetooth bluez python3 python3-venv
sudo systemctl enable --now bluetooth
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install '.[all]'
```

Sul Pi Zero usare `pip install .` per il solo SwitchBot, `.[sensors]` per i decoder BLE o `.[cloud]` per SwitchBot e Tuya. Verificare con `ble-sensors-mqtt --list-plugins`.

3. Plugin supportati

Nome	Famiglia	Accesso
switchbot	SwitchBot	BLE locale
xiaomi	Xiaomi/Mijia/HHCC	BLE, eventuale bind key
govee	Govee	BLE locale
inkbird	Inkbird	BLE/connessione su modelli compatibili
thermopro	ThermoPro	BLE locale
qingping	Qingping/ClearGrass	BLE locale
bthome	BTHome, Shelly BLU, ATC/PVWX-BTHome	BLE locale
ruuvi	RuuviTag	BLE locale
sensornpush	SensorPush	BLE locale
airthings	Airthings	BLE GATT attivo
mopeka	Mopeka	BLE locale
tuya-cloud	Tuya/Smart Life	HTTPS cloud

Il supporto dipende da modello e firmware.

4. Scan, selezione e alias

```
.venv/bin/ble-sensors-mqtt --scan --scan-duration 15
.venv/bin/ble-sensors-mqtt --scan --plugin switchbot --plugin bthome
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 \
  --device AA:BB:CC:DD:EE:FF \
  --device-name 'AA:BB:CC:DD:EE:FF=Sala Server'
```

Lo scan non richiede MQTT. Per Tuya o sotto-canali usare `--sensor-name 'ID=NOME'`. L'alias è propagato a ogni output e il nome BLE originale resta in `bluetooth_name`.

Per Xiaomi/BTHome cifrati, salvare la bind key esadecimale in un file protetto e aggiungere al TOML: `[ble_keys."MAC"], plugin="xiaomi" o "bthome", e bindkey_file="/percorso/file"`. Non passare mai la chiave in CLI.

5. Esecuzione e MQTT

Home Assistant MQTT Discovery

Aggiungere `--home-assistant-discovery` per pubblicare la configurazione retained di Home Assistant MQTT Discovery. Il prefisso discovery predefinito è `homeassistant`; può essere cambiato con `--home-assistant-discovery-prefix`. Ogni sensore esportato diventa un device Home Assistant, con una entità sensor per ogni valore scalare del payload più le entità diagnostiche RSSI e protocollo. Lo stato resta sul normale topic MQTT del gateway e l'availability è legata a `<mqtt-prefix>/bridge/status`.

```
.venv/bin/ble-sensors-mqtt --mqtt-host 127.0.0.1 --home-assistant-discovery
```

La configurazione discovery è retained. I topic discovery obsoleti vengono rimossi esplicitamente con payload retained vuoti, così sensori/entità rimossi scompaiono da Home Assistant. Nei deployment persistenti/systemd mantenere attivo il file di stato runtime per consentire il cleanup anche dopo i riavvii.

```
.venv/bin/ble-sensors-mqtt --mqtt-host 127.0.0.1 --once
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --poll-interval 60 --allow-insecure-mqtt
.venv/bin/ble-sensors-mqtt --mqtt-host mqtt.example.it --mqtt-port 8883 \
--mqtt-username sensor-publisher \
--mqtt-password-file /etc/ble-sensors-mqtt/mqtt-password --mqtt-tls
```

Default: polling ogni 30 secondi. Il certificato TLS è sempre verificato. I file segreti devono essere 0640 o più restrittivi. Ogni payload contiene `manufacturer`, `model`, `protocol` e, in `data`, tutte le informazioni del decoder.

6. Tuya Cloud

Usare `--cloud-help`, creare un progetto Smart Home su Tuya IoT e collegare l'account Smart Life/Tuya. Conservare Access ID e Access Secret in file distinti protetti.

```
[cloud.tuya]
enabled = true
region = "eu"
api_key_file = "/etc/ble-sensors-mqtt/tuya-api-key"
api_secret_file = "/etc/ble-sensors-mqtt/tuya-api-secret"
api_device_id = "DEVICE_ID_DI_RIFERIMENTO"
device_ids = ["DEVICE_ID_1"]
```

Il TOML deve essere 0640 o più restrittivo. Provare con:

```
.venv/bin/ble-sensors-mqtt --scan --cloud-config /etc/ble-sensors-mqtt/cloud.toml
```

Un guasto Tuya non interrompe la raccolta BLE. Valutare privacy e dipendenza dal cloud. I dispositivi Tuya con firmware BTHome possono funzionare localmente.

7. Prometheus

Abilitare l'exporter HTTP con:

```
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --prometheus
curl http://127.0.0.1:9105/metrics
```

Il listener predefinito è `127.0.0.1:9105`. Un bind non-loopback richiede il consenso esplicito `--allow-external-prometheus`. L'endpoint non implementa autenticazione o TLS: per accesso esterno usare firewall, VPN oppure un reverse proxy autenticato.

Metriche Prometheus esposte

Metrica	Tipo	Label	Significato
ble_sensors_up	gauge	address, name, manufacturer, model, protocol	1 per snapshot fresco, 0 quando lo snapshot esportato è riusato/stale
ble_sensors_stale	gauge	stesse label sensore	1 se lo snapshot proviene da un ciclo precedente, altrimenti 0
ble_sensors_temperature_celsius	gauge	stesse label sensore	valore numerico temperature trovato ricorsivamente, in gradi Celsius
ble_sensors_humidity_percent	gauge	stesse label sensore	valore numerico humidity trovato ricorsivamente, in percentuale
ble_sensors_battery_percent	gauge	stesse label sensore	valore numerico battery o battery_percent trovato ricorsivamente
ble_sensors_rssi_dbm	gauge	stesse label sensore	RSSI Bluetooth top-level in dBm, se disponibile
ble_sensors_sensor_value	gauge	label sensore + key, unit	ogni scalare numerico; i booleani sono esportati come 0/1
ble_sensors_sensor_info	gauge	label sensore + key, unit, value	ogni stringa non nulla, rappresentata da un campione costante pari a 1
ble_sensors_devices	gauge	nessuna	numero di snapshot attualmente esportati, inclusi eventuali snapshot stale riusati
ble_sensors_cycles_total	counter	nessuna	cicli di polling tentati dal gateway
ble_sensors_cycles_failed_total	counter	nessuna	cicli di polling marcati come falliti

Le metriche generiche derivano esclusivamente dall'oggetto normalizzato data del sensore. Dizionari annidati vengono appiattiti con path puntati, ad esempio `air.co2`; elementi di liste/tuple usano indici numerici come `channels.0`. La mappa riservata `data.units` non viene esportata come dato: quando presente, fornisce la label `unit` in base al nome della foglia. Vengono esportati al massimo 256 scalari per sensore. I valori `null` non vengono esportati in Prometheus.

Le metriche dedicate temperatura/umidità/batteria e la metrica generica possono intenzionalmente esporre la stessa misura. I nomi dedicati semplificano dashboard stabili, mentre la famiglia generica preserva tutti i valori scalari forniti dai plugin. Poiché `ble_sensors_sensor_info` inserisce le stringhe nelle label, stringhe molto variabili possono aumentare la cardinalità Prometheus.

Esempio:

```
ble_sensors_up{address="AA:BB:CC:DD:EE:FF",name="Sala",manufacturer="SwitchBot",model="Meter Plus",protocol="SwitchBot"}
ble_sensors_temperature_celsius{address="AA:BB:CC:DD:EE:FF",name="Sala",manufacturer="SwitchBot",model="Meter Plus",protocol="SwitchBot"}
ble_sensors_sensor_value{address="AA:BB:CC:DD:EE:FF",name="Sala",manufacturer="SwitchBot",model="Meter Plus",protocol="SwitchBot",key="temp",unit="C"}
ble_sensors_sensor_info{address="AA:BB:CC:DD:EE:FF",name="Sala",manufacturer="SwitchBot",model="Meter Plus",protocol="SwitchBot",key="temp",unit="C",value="25.0"}
ble_sensors_humidity_percent{address="AA:BB:CC:DD:EE:FF",name="Sala",manufacturer="SwitchBot",model="Meter Plus",protocol="SwitchBot",key="humid",unit="%"}
ble_sensors_battery_percent{address="AA:BB:CC:DD:EE:FF",name="Sala",manufacturer="SwitchBot",model="Meter Plus",protocol="SwitchBot",key="battery",unit="%"}
ble_sensors_rssi_dbm{address="AA:BB:CC:DD:EE:FF",name="Sala",manufacturer="SwitchBot",model="Meter Plus",protocol="SwitchBot",key="rssi",unit="dBm"}
ble_sensors_devices
ble_sensors_cycles_total
ble_sensors_cycles_failed_total
```

Lo stesso server HTTP espone anche `/healthz` e `/readyz`. `/healthz` indica la liveness del processo; `/readyz` restituisce HTTP 503 finché non completa con successo un ciclo e nuovamente quando l'ultimo ciclo riuscito diventa troppo vecchio.

```
--prometheus --prometheus-host 0.0.0.0 --prometheus-port 9200 --allow-external-prometheus
```

8. SNMP

Abilitare l'agente SNMPv2c read-only con un file community protetto:

```
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --snmp \
--snmp-community-file /etc/ble-sensors-mqtt/snmp-community
```

Il listener predefinito è 127.0.0.1:1161/udp. Il bind non-loopback richiede --allow-external-snmp. SNMPv2c non cifra community o payload: limitare l'accesso UDP oppure usare una VPN/proxy SNMPv3.

Con il default --snmp-base-oid 1.3.6.1.4.1.32473.1.1, l'albero esportato è:

Suffisso OID	Oggetto MIB	Tipo	Significato
.1.0	bleSensorsVersion	DisplayString	identificazione/versione applicazione
.2.0	bleSensorsDeviceCount	Gauge32	numero di snapshot attualmente esportati
.10.1.1.I	bleSensorsIndex	Integer32	indice transitorio riga dispositivo
.10.1.2.I	bleSensorsAddress	DisplayString	identificatore/indirizzo normalizzato sensore
.10.1.3.I	bleSensorsName	DisplayString	nome/alias effettivo del sensore
.10.1.4.I	bleSensorsRssi	Integer32	RSSI in dBm, omesso se non disponibile
.10.1.5.I	bleSensorsTemperatureMilliCelsius	Integer32	temperatura moltiplicata per 1000
.10.1.6.I	bleSensorsHumidityMilliPercent	Gauge32	umidità relativa moltiplicata per 1000
.10.1.7.I	bleSensorsBatteryPercent	Gauge32	percentuale batteria arrotondata all'intero
.10.1.8.I	bleSensorsObservedAt	DisplayString	timestamp originale dell'osservazione
.10.1.9.I	bleSensorsManufacturer	DisplayString	produttore, oppure Unknown
.10.1.10.I	bleSensorsModel	DisplayString	modello, oppure Unknown
.10.1.11.I	bleSensorsProtocol	DisplayString	protocollo, oppure Unknown
.10.1.12.I	bleSensorsStale	Gauge32	1 per dato riusato/stale, altrimenti 0
.20.1.1.I.J	bleSensorsValueKey	DisplayString	chiave/path scalare appiattito
.20.1.2.I.J	bleSensorsValue	DisplayString	scalare reso come testo
.20.1.3.I.J	bleSensorsValueType	DisplayString	null, boolean, number o string
.20.1.4.I.J	bleSensorsValueUnit	DisplayString	unità da data.units, se disponibile
.20.1.5.I.J	bleSensorsValueDeviceIndex	Integer32	indice dispositivo I

Suffisso OID	Oggetto MIB	Tipo	Significato
.20.1.6.I.J	bleSensorsValueIndex	Integer32	indice scalare J

Le righe .10 sono ordinate per identificatore normalizzato del sensore: l'indice I è quindi transitorio e può cambiare quando cambia l'insieme dei dispositivi esportati. Le colonne opzionali delle misure comuni non esistono quando il valore non è disponibile. La tabella .20 contiene fino a 256 valori scalari per ogni oggetto data e usa le stesse regole di flattening di Prometheus. I campi testuali SNMP sono limitati dall'agente a 512 byte codificati.

La MIB testuale inclusa è docs/ble-sensors-mqtt-mib.txt. Il PEN 32473 è solo documentativo. In produzione usare un enterprise OID assegnato con --snmp-base-oid. Cambiando il base OID viene spostato lo stesso layout di suffissi mostrato sopra; la MIB testuale inclusa continua invece a nominare la radice documentativa predefinita.

Esempi di walk:

```
snmpwalk -v2c -c COMMUNITY_CASUALE 127.0.0.1:1161 1.3.6.1.4.1.32473.1.1
snmpwalk -v2c -c COMMUNITY_CASUALE 127.0.0.1:1161 1.3.6.1.4.1.32473.1.1.10
snmpwalk -v2c -c COMMUNITY_CASUALE 127.0.0.1:1161 1.3.6.1.4.1.32473.1.1.20
```

9. systemd

Per una installazione completa a partire dal clone GitHub (prerequisiti host, controllo Bluetooth, envv e systemd):

```
git clone https://github.com/desalvo/ble-sensors-mqtt.git
cd ble-sensors-mqtt
sudo scripts/install-from-github.sh --mqtt-host mqtt.example.net --prometheus
```

Usare l'installer systemd direttamente quando il clone e i prerequisiti sono già presenti:

```
# Interattivo: i valori CLI sono i default mostrati nelle domande
sudo scripts/install-systemd.sh --mqtt-host mqtt.example.net --prometheus

# Provisioning completamente non interattivo
sudo scripts/install-systemd.sh --non-interactive \
  --mqtt-host mqtt.example.net --mqtt-tls \
  --mqtt-username ble-sensors-publisher --mqtt-password-file ./mqtt-password \
  --home-assistant-discovery --prometheus
```

L'installer crea utente di servizio, envv isolata, configurazione root-owned e /etc/ble-sensors-mqtt/config.toml; le opzioni ripetibili sono salvate come array TOML e i valori con spazi sono preservati senza parsing shell. In modalità interattiva i valori CLI restano i default mostrati dal wizard e le opzioni ripetibili già passate vengono mantenute. Con --non-interactive l'installazione usa esclusivamente valori CLI/default. Vedere docs/SYSTEMD.it.md.

10. Docker e Kubernetes

```
# Build multiarch + push Docker Hub di desalvo/ble-sensors-mqtt:<VERSION>
docker login
scripts/build-docker.sh

# Demone continuo Docker
cd docker
cp .env.example .env
cp arguments.example arguments
docker compose up -d

# Demone continuo Kubernetes
kubectl label node NOME_NODO ble-sensors-mqtt/bluetooth=true
kubectl apply -k kubernetes/
```

Prima di Docker/Kubernetes verificare l'host con sudo scripts/check-bluetooth-host.sh --strict. Entrambi montano /run/dbus read-only e usano BlueZ dell'host via system D-Bus; non richiedono modalità privilegiata nel modello supportato. Kubernetes include anche kubernetes/bluetooth-test-pod.yaml per validare --scan sul nodo.

L'immagine supporta linux/amd64 e linux/arm64. Docker/Kubernetes montano il system D-Bus host per BlueZ, persistono lo stato runtime, inviano MQTT in uscita ed espongono Prometheus TCP/9105 e SNMP UDP/1161. Manifest LoadBalancer Kubernetes opzionali espongono Prometheus e SNMP all'esterno solo se applicati esplicitamente. scripts/build-docker.sh esegue di default il push di

desalvo/ble-sensors-mqtt:<VERSION>; la CI può pubblicare l'immagine versionata sui tag vX.Y.Z e latest su main. Vedere docs/DOCKER.it.md e docs/KUBERNETES.it.md; gli endpoint di monitoring esterni vanno limitati a reti fidate.

11. Estensioni

Un plugin esterno implementa `SensorPlugin.decode()` e restituisce `SensorReading`; si registra nel gruppo entry point `ble_sensors_mqtt.sensor_plugins`. Ogni lettura valorizza marca, modello e protocollo. Le eccezioni dei plugin sono isolate.

12. Sicurezza e diagnosi

- non eseguire come root;
- proteggere file MQTT, Tuya e SNMP;
- usare ACL MQTT minime e TLS;
- non esporre Prometheus/SNMP senza controllo di rete;
- aggiornare e verificare dipendenze con `pip-audit`;
- provare i modelli reali prima della produzione.

```
bluetoothctl show
.venv/bin/ble-sensors-mqtt --list-plugins
.venv/bin/ble-sensors-mqtt --scan --scan-duration 20 --log-level DEBUG
journalctl -u ble-sensors-mqtt --since today
```

La verifica AGID è un'autovalutazione tecnica, non una certificazione.

13. Build e rollback

`scripts/build-package.sh` genera build UTC YYYYMMDD-HHMM, ZIP, TAR.GZ e SHA-256. Conservare il pacchetto precedente per il rollback e non modificare direttamente `site-packages`.

14. Health di produzione e gate di release

Health e readiness

Quando Prometheus è abilitato, lo stesso server HTTP espone:

- `/metrics` per Prometheus;
- `/healthz` per la liveness del processo;
- `/readyz` per la readiness. Restituisce HTTP 503 fino al primo ciclo riuscito e nuovamente se il polling riuscito diventa troppo vecchio.

L'esposizione non-loopback di Prometheus/health richiede `--allow-external-prometheus`; SNMPv2c non-loopback richiede `--allow-external-snmp`. MQTT remoto o con credenziali in chiaro richiede `--allow-insecure-mqtt`. Questi override sono consensi espliciti al rischio, non default consigliati.

`--plugin-timeout` limita una singola chiamata decoder/cloud (default 15 secondi). `--stale-cycles` stabilisce dopo quanti cicli consecutivi mancanti rimuovere lo stato MQTT retained (default 3). `--state-file` persiste lo stato tra riavvii; l'unità systemd usa `/var/lib/ble-sensors-mqtt/state.json`.

Verifica release

Installare `.[all,dev,release]` ed eseguire:

```
scripts/release-check.sh
```

Il gate richiede lint, test/coverage, Bandit, audit dipendenze, generazione PDF EN/IT, build wheel e sdist, twine check, SBOM CycloneDX e verifica del pacchetto di release. La CI ripete i test runtime su Python 3.11, 3.12 e 3.13.

Per la pubblicazione su GitHub, eseguire prima il push del commit sorgente e poi il push di un tag vX.Y.Z corrispondente. Il workflow del tag esegue tutti i gate, costruisce gli artefatti deterministici di release, crea checksum

SHA-256 consolidati e pubblica automaticamente la GitHub Release soltanto dopo il successo dei job di test e build. Il job di release verifica che la versione del tag corrisponda a `VERSION` e `pyproject.toml`.

Fallback stale e cache MQTT persistente

Usare `--reuse-stale-data` per mantenere l'ultimo snapshot in memoria quando un ciclo non rileva il sensore o non produce dati. Gli snapshot riusati conservano il timestamp originale e impostano `stale: true`; quelli freschi impostano `stale: false`. Lo spool MQTT SQLite è attivo per default, ha limite logico 1 GiB, continua ad accodare durante l'indisponibilità del broker, viene svuotato FIFO alla riconnessione e rimuove i record solo dopo ACK MQTT. Configurare con `--mqtt-cache-path PATH`, `--mqtt-cache-max-size SIZE` o `--no-mqtt-cache`.

Frontend web opzionale

Abilitare la console web autenticata con `--frontend` dopo aver installato `. [web]` oppure `. [all]`. Il bind default è `127.0.0.1:8080`; usare `--allow-external-frontend` per indirizzi non-loopback e proteggere gli accessi remoti con HTTPS o reverse proxy TLS. Le credenziali iniziali sono `admin / password`, con cambio password obbligatorio al primo login. La console offre stato sensori responsive desktop/mobile, autorizzazione admin/reader, configurazione runtime persistente, autenticazione locale/LDAP/OIDC, MFA TOTP per utenti locali/LDAP, gestione utenti e import/export cifrato di configurazione e cache/stato disponibili. Dettagli in `docs/FRONTEND.it.md`.